FIXKO - Database Data Dictionary

Database: fixko (MySQL / phpMyAdmin)
Generated: Wednesday, October 28, 2026

Tables covered

- users
- rooms
- reports
- comments
- ticket_status_histories
- feedback
- notifications
- password_reset_tokens
- personal_access_tokens
- cache
- cache_locks
- sessions
- migrations

Each table below is shown with its fields (columns), the data type of each field, what the field is used for, and the constraints (rules the database enforces, such as Primary Key, Unique, Required, Nullable, Default values, and Foreign Key references). A short explanation of the table's purpose follows under every table.

Table: users

Table	Field Name	Data Type	Description	Constraints
users	id	BIGINT (integer)	Unique user ID	Primary Key - Auto-increment
users	name	VARCHAR (255)	Full display name	Required
users	first_name	VARCHAR (60)	First name (optional split of name)	Nullable
users	last_name	VARCHAR (60)	Last name (optional split of name)	Nullable
users	middle_name	VARCHAR (60)	Middle name (optional split of name)	Nullable
users	email	VARCHAR (255)	Login email - must be school email	Unique - Required
users	school_id	VARCHAR (255)	Student / staff school ID number	Unique - Required
users	password	VARCHAR (255)	Hashed password	Required (hashed)
users	role	ENUM (user / maintenance / admin)	Account role	Default: user
users	department	VARCHAR (255)	Department of the user	Nullable
users	email_verified_at	TIMESTAMP	When email OTP was verified	Nullable
users	account_status	VARCHAR (20)	Account state: active / suspended / restricted	Default: active
users	status_until	TIMESTAMP	When a suspension ends (end of ban)	Nullable
users	warnings_count	INT (unsigned)	Counts how many warnings the user got	Default: 0
users	address	VARCHAR (255)	Home address of the user	Nullable
users	gender	ENUM (male / female / other)	Gender of the user	Nullable
users	block_section	VARCHAR (120)	Block or section of a student	Nullable
users	deleted_at	TIMESTAMP	Soft delete - moves account to recycle bin	Nullable
users	remember_token	VARCHAR (255)	Keeps user logged in on one device	Nullable
users	created_at	TIMESTAMP	Account creation date	Auto-generated
users	updated_at	TIMESTAMP	Last time the account was edited	Auto-generated

About this table

Holds every account of people who use FIXKO - students, admin staff, and maintenance personnel. It stores their identity (name, school ID, email), their login password (hashed so nobody can read it), and their assigned role (user, admin, or maintenance). It also tracks moderation: the account status (active, suspended, locked), how long a restriction lasts, how many warnings the user received, and profile details like address, gender, and block section. Deleted accounts are never erased immediately - deleted_at keeps them in the Recycle Bin so an admin can restore them.

Table: rooms

Table	Field Name	Data Type	Description	Constraints
rooms	id	BIGINT (integer)	Unique room ID	Primary Key - Auto-increment
rooms	building	VARCHAR (255)	Name of the campus building	Required
rooms	floor	VARCHAR (255)	Floor number of the room	Nullable
rooms	name	VARCHAR (255)	Room name or number	Required
rooms	code	VARCHAR (255)	Short room code used in QR codes	Unique - Nullable
rooms	department	VARCHAR (255)	Department owning the room	Nullable
rooms	created_at	TIMESTAMP	Room creation date	Auto-generated
rooms	updated_at	TIMESTAMP	Last time the room was edited	Auto-generated

About this table

The campus directory. It lists every building of the school together with its floors and individual rooms (given names plus short codes like 'CMA BUILDING-Room 203'). Room codes are unique so they can be stamped into QR codes posted in rooms; scanning a QR code lets users start a report pointed at exactly that room. Reports link to this table to say where a problem is located.

Table: reports

Table	Field Name	Data Type	Description	Constraints
reports	id	BIGINT (integer)	Unique report ID (ticket code FIX-XXXXX)	Primary Key - Auto-increment
reports	reporter_id	BIGINT (integer)	User who submitted the report	Foreign Key: users.id - Required - Deletes with user
reports	reporter_school_id_snapshot	VARCHAR (255)	Copy of reporter's school ID number at report time	Required (copy)
reports	reporter_email_snapshot	VARCHAR (255)	Copy of reporter's email at report time	Required (copy)
reports	room_id	BIGINT (integer)	Room where the problem is	Foreign Key: rooms.id - Required
reports	equipment_id	BIGINT (integer)	Old field - equipment was removed so this stays empty	Nullable (no longer used)
reports	category	VARCHAR (255)	Problem category	Nullable
reports	item_name	VARCHAR (255)	Name of the item being reported	Nullable
reports	condition	VARCHAR (255)	Condition of the item when reported	Nullable
reports	quantity	INT (unsigned)	Number of items affected	Nullable
reports	description	TEXT	Details of the problem	Required
reports	priority	ENUM (low / medium / high / critical)	How urgent the problem is	Default: medium
reports	status	ENUM (pending / under_review / in_progress / under_maintenance / resolved)	Progress of the report	Default: pending
reports	is_flagged	TINYINT (boolean)	Marked by admin for moderation	Default: false
reports	flag_reason	VARCHAR (100)	Reason why the report was flagged	Nullable
reports	flag_note	TEXT	Extra note about the flag	Nullable
reports	flagged_at	TIMESTAMP	When the report was flagged	Nullable
reports	handled_by	BIGINT (integer)	Maintenance worker assigned to fix it	Foreign Key: users.id - Nullable
reports	photo_url	VARCHAR (255)	Path of the uploaded photo	Nullable
reports	resolved_at	TIMESTAMP	When the report was marked resolved	Nullable
reports	deleted_at	TIMESTAMP	Soft delete - moves report to recycle bin	Nullable
reports	deleted_reason	VARCHAR (30)	Why the report was deleted	Nullable

Table	Field Name	Data Type	Description	Constraints
reports	created_at	TIMESTAMP	Report creation date	Auto-generated
reports	updated_at	TIMESTAMP	Last time the report was edited	Auto-generated

About this table

The heart of FIXKO. Every time someone reports a broken or damaged facility, one row is created here describing the problem, its urgency level (low to critical), its status as it moves through progress (pending, under review, in progress, under maintenance, resolved), the room it belongs to, and - once an admin assigns it - the maintenance worker handling it. A photo of the problem can be attached. Admin moderation fields (is_flagged etc.) keep an eye on misuse, and deleted_at + deleted_reason send removed reports to the Recycle Bin instead of destroying them. Note: reports still has an old equipment_id column from a removed feature; it is kept but is no longer used.

Table: comments

Table	Field Name	Data Type	Description	Constraints
comments	id	BIGINT (integer)	Unique comment ID	Primary Key - Auto-increment
comments	report_id	BIGINT (integer)	Report the comment belongs to	Foreign Key: reports.id - Required
comments	author_id	BIGINT (integer)	User who wrote the comment	Foreign Key: users.id - Required
comments	message	TEXT	Comment text	Required
comments	created_at	TIMESTAMP	Comment creation date	Auto-generated
comments	updated_at	TIMESTAMP	Last time the comment was edited	Auto-generated

About this table

A simple message thread under each report. Students, admins, and maintenance workers can exchange short messages on a report - asking for details, giving updates, or coordinating a visit. Comments are permanently tied to their report (they disappear only if the report itself is deleted) and remember who wrote each message.

Table: ticket_status_histories

Table	Field Name	Data Type	Description	Constraints
ticket_status_histories	id	BIGINT (integer)	Unique history entry ID	Primary Key - Auto-increment
ticket_status_histories	report_id	BIGINT (integer)	Report this history belongs to	Foreign Key: reports.id - Required
ticket_status_histories	old_status	VARCHAR (255)	Status before the change	Nullable
ticket_status_histories	new_status	VARCHAR (255)	Status after the change	Required
ticket_status_histories	changed_by	BIGINT (integer)	User who changed the status	Foreign Key: users.id - Required
ticket_status_histories	changed_at	TIMESTAMP	When the status was changed	Auto-generated

About this table

An audit trail (log book) of a report's life. Each time someone changes a report's status, one row is written here recording the old status, the new status, who made the change, and when. This gives the full timeline that appears on the report detail page, so it can always be proven who did what and when.

Table: feedback

Table	Field Name	Data Type	Description	Constraints
feedback	id	BIGINT (integer)	Unique feedback ID	Primary Key - Auto-increment
feedback	report_id	BIGINT (integer)	Report being rated	Foreign Key: reports.id - Required - Unique (one per report)
feedback	user_id	BIGINT (integer)	User who gave the rating	Foreign Key: users.id - Required
feedback	rating	TINYINT (1-5)	Star rating for the fix	Required
feedback	comment	TEXT	Extra written feedback	Nullable
feedback	created_at	TIMESTAMP	Feedback creation date	Auto-generated
feedback	updated_at	TIMESTAMP	Last time feedback was edited	Auto-generated

About this table

Closing the loop. Once a report is resolved, the original reporter star-rates how well the problem was fixed and may add a short comment. Only one feedback record can exist per report (uniqueness is enforced), which is why the API can show a single rating per report in summaries and analytics.

Table: notifications

Table	Field Name	Data Type	Description	Constraints
notifications	id	BIGINT (integer)	Unique notification ID	Primary Key - Auto-increment
notifications	user_id	BIGINT (integer)	User who receives the notification	Foreign Key: users.id - Required
notifications	message	TEXT	Notification text	Required
notifications	is_read	TINYINT (boolean)	Read marker	Default: false
notifications	archived_at	TIMESTAMP	Archive marker (hidden from list)	Nullable
notifications	related_report_id	BIGINT (integer)	Report the notification is about	Foreign Key: reports.id - Nullable
notifications	created_at	TIMESTAMP	Notification creation date	Auto-generated
notifications	updated_at	TIMESTAMP	Last time the notification was edited	Auto-generated

About this table

Each user's notification inbox. Whenever something happens to a user - their report changes status, a maintenance worker is assigned, or the admin sends a warning - a row is inserted here with the message text and a link to the related report. Each notification tracks whether it was read (is_read) and whether it was archived away by the user (archived_at).

Table: password_reset_tokens

Table	Field Name	Data Type	Description	Constraints
password_reset_tokens	email	VARCHAR (255)	Email address asking for a reset	Primary Key
password_reset_tokens	token	VARCHAR (255)	One-time reset code sent by email	Required
password_reset_tokens	created_at	TIMESTAMP	When the token was made - expires old ones	Nullable

About this table

Works as temporary storage for password recovery. When a user forgets their password, a one-time reset code is generated and stored here together with the user's email. After the code is used (or replaced by a newer request), it is overwritten. It exists so a reset link can be verified without exposing anything in the users table itself.

Table: personal_access_tokens

Table	Field Name	Data Type	Description	Constraints
personal_access_tokens	id	BIGINT (integer)	Unique token ID	Primary Key - Auto-increment
personal_access_tokens	tokenable_type	VARCHAR (255)	Type of account the token belongs to (users)	Required
personal_access_tokens	tokenable_id	BIGINT (integer)	ID of the account the token belongs to	Required
personal_access_tokens	name	VARCHAR (255)	Name of the device or login session	Required
personal_access_tokens	token	VARCHAR (64)	The login token itself (hashed)	Unique - Required
personal_access_tokens	abilities	TEXT	What the token is allowed to do	Nullable
personal_access_tokens	last_used_at	TIMESTAMP	When the token was last used	Nullable
personal_access_tokens	expires_at	TIMESTAMP	When the token expires	Nullable
personal_access_tokens	created_at	TIMESTAMP	Token creation date	Auto-generated
personal_access_tokens	updated_at	TIMESTAMP	Last time the token was edited	Auto-generated

About this table

Created by Laravel Sanctum to manage login sessions for the API. Every time a user logs in, the app issues a secret token stored here; the browser presents it on later requests so the user stays logged in without resending the password. Each token remembers which account it belongs to, which device it belongs to, and when it was last used or when it expires.

Table: cache

Table	Field Name	Data Type	Description	Constraints
cache	key	VARCHAR (255)	Name of the stored cache item	Primary Key
cache	value	MEDIUMTEXT	The cached data	Required
cache	expiration	INT	Cached data expires (fails) at this date	Required

About this table

A technical table used by the Laravel framework to temporarily store small pieces of data (like computed results) so pages load faster. Content is short-lived; each entry has an expiration time after which it is forgotten. It is part of the framework, not part of FIXKO's business data.

Table: cache_locks

Table	Field Name	Data Type	Description	Constraints
cache_locks	key	VARCHAR (255)	Name of the lock	Primary Key
cache_locks	owner	VARCHAR (255)	Which process holds the lock	Required
cache_locks	expiration	INT	When the lock expires	Required

About this table

A small technical table that prevents two background processes from grabbing the same cache task at the same moment. The 'owner' column records which process holds the lock. Part of the Laravel framework.

Table: sessions

Table	Field Name	Data Type	Description	Constraints
sessions	id	VARCHAR (255)	Unique session ID	Primary Key
sessions	user_id	BIGINT (integer)	Logged-in user of this session	Foreign Key: users.id - Nullable
sessions	ip_address	VARCHAR (45)	IP address of the device	Nullable
sessions	user_agent	TEXT	Browser and device info	Nullable
sessions	payload	TEXT	Stored session data	Required
sessions	last_activity	INT	Last time the user was active	Required

About this table

Server-side login sessions. When used, it remembers who is logged in, from which computer (IP address and browser information), and when they were last active. It lets the system cleanly expire stale sessions. Part of the Laravel framework.

Table: migrations

Table	Field Name	Data Type	Description	Constraints
migrations	id	INT (integer)	Unique row ID	Primary Key - Auto-increment
migrations	migration	VARCHAR (255)	Name of the applied migration file	Required
migrations	batch	INT	Which setup run (batch) applied it	Required

About this table

The database's own notebook. It lists every migration (database setup script) that has already been executed and in which batch, so re-running migrations never repeats work. Essential for keeping the schema reproducible, but holds no application data.